Lab 3: Monte Carlo Methods

Morgan Cooper
MSDS 684 — Reinforcement Learning

March 27, 2026

Section 1: Project Overview

This lab builds upon earlier concepts like Markov Decision Processes (MDP) and Dynamic Programming (DP) by introducing Monte Carlo Simulations. Specifically, this lab uses Monte Carlo (MC) and Reinforcement Learning (RL) to learn to play blackjack. This is done by letting an agent play blackjack hands hundreds of thousands of times without knowning any rules, just observing the outcomes. MC RL follows a model-free approach, since the agent has zero initial knowledge of card probabilities, or dealer behavior. This contrasts well with DP since DP requires a complete model of the environment. This lab investigates the first-visit MC control with differeing epsilon policies.

MC methods fit value functions by estimating full rewards at the end of each episode, but only uses the first occurence of each state-action pair per episode, ensuring return samples are independent and unbiased (Sutton Barto, 2020). To balance exploration and exploitation RL with MC uses an epsilon greedy policy, which results in a a high chance of non-greedy actions at the beginning of training and decays to a very small chance of non-greedy actions late in training. This ensures the agent has explored all actions sufficiently to determine an optimal policy. This lab also uses on-policy MC control meaning the same epsilon-greedy policy that generates episodes is also the policy being improved.

The environment used in the lab follows:

Blackjack-v1 (Gymnasium):

- State space: discrete 3-tuple (player sum 4–21, dealer showing card 1–10, usable ace True/False) — approximately 400 unique states
- Action space: 2 discrete actions — hit (draw a card) or stick (stop drawing)
- Rewards: $+1$ for winning, -1 for losing, 0 for a draw — only awarded at the end of each hand
- Terminal conditions: player busts (exceeds 21), player sticks, or player receives a natural blackjack
- Uses Sutton & Barto rules: infinite deck, natural blackjack only counts without hitting

I expect the agent to sucessfully learn an optimal blackjack policy, but only if its given enough training time. Since I am familiar with Blackjack I am expecting the agent to learn quickly to stick when the value of the hand is high, and hit when it is low. In blackjack, if the player busts first the dealer doesnt have to reveal their cards, even if they too would have busted the player looses. Therfore, I dont expect win rate to get higher than 45

Section 2: Deliverables

GitHub Repository

`GitHub Repository: https://github.com/cooper-rm/monte-carlo-methods`

Implementation Summary

[PLACEHOLDER: Write your implementation summary here]

Key Results & Analysis

[PLACEHOLDER: Write your key results and analysis here]

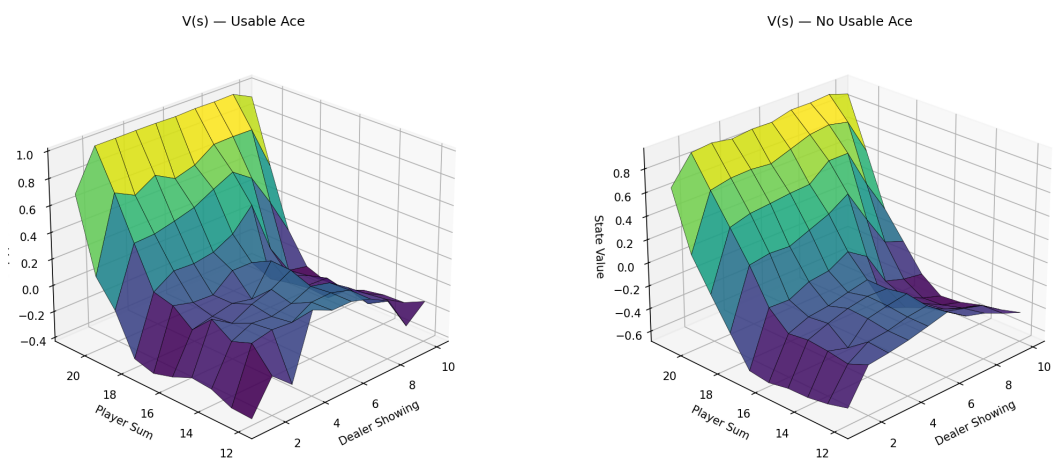

Figure 1: [PLACEHOLDER: Interpretive caption for figure 1]

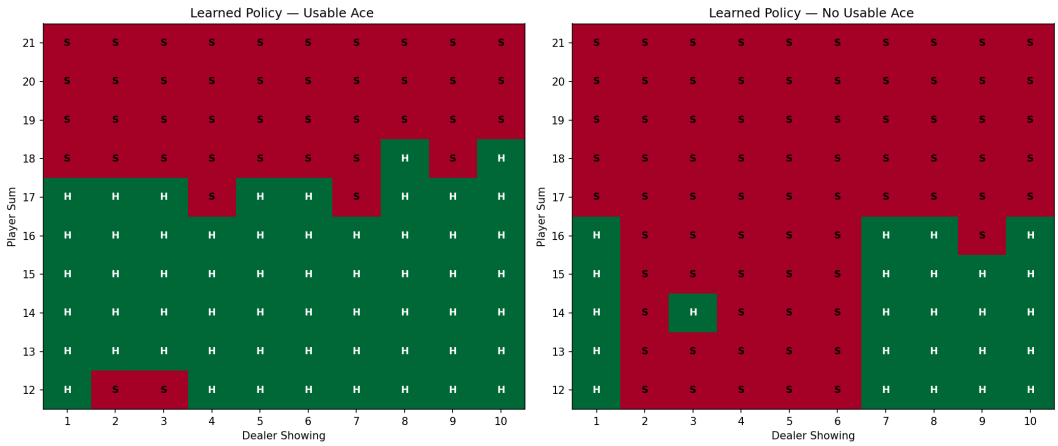

Figure 2: [PLACEHOLDER: Interpretive caption for figure 2]

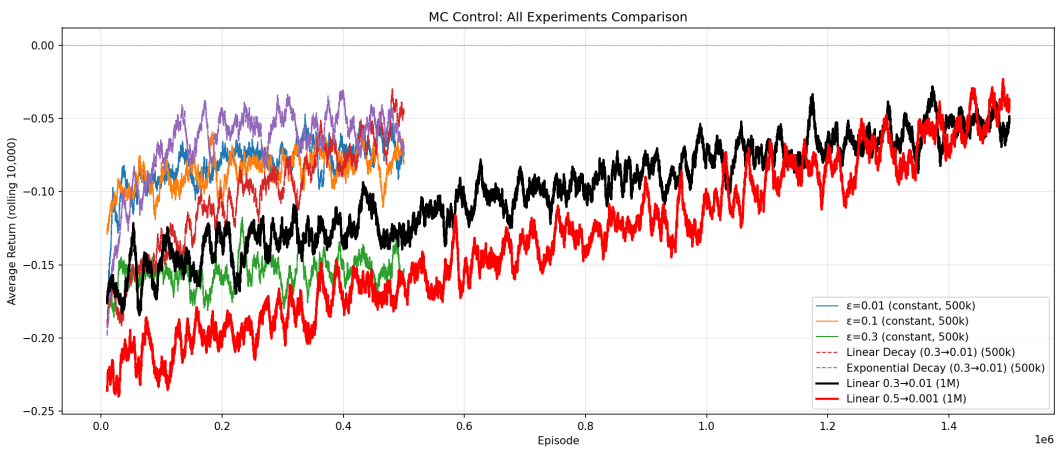

Figure 3: [PLACEHOLDER: Interpretive caption for figure 3]

Section 3: AI Use Reflection

Initial Interaction

[PLACEHOLDER: Write your initial interaction here]

Iteration Cycle

Iteration 1: [PLACEHOLDER: Title]

[PLACEHOLDER: Describe the first debugging cycle]

Iteration 2: [PLACEHOLDER: Title]

[PLACEHOLDER: Describe the second debugging cycle]

Iteration 3: [PLACEHOLDER: Title]

[PLACEHOLDER: Describe the third debugging cycle]

Critical Evaluation

[PLACEHOLDER: Write your critical evaluation here]

Learning Reflection

[PLACEHOLDER: Write your learning reflection here]

Section 4: Speaker Notes

- **Problem:** [PLACEHOLDER]
- **Method:** [PLACEHOLDER]
- **Design choice:** [PLACEHOLDER]
- **Key result:** [PLACEHOLDER]
- **Insight:** [PLACEHOLDER]
- **Challenge:** [PLACEHOLDER]
- **Connection:** [PLACEHOLDER]

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
2. Anthropic. (2025). Claude Code [Large language model CLI tool]. `https://claude.ai`
3. OpenAI. (2025). ChatGPT (GPT-4o) [Large language model]. `https://chat.openai.com`